

PRAVAH: A 2-Wide Out-of-Order Superscalar RISC-V Processor

Final Report — Seasons of Code 2026, IIT Bombay

By: Ashwini Kumar
2581298

July 2026

1. Introduction

Every scalar, in-order pipelined processor — the kind covered in a first computer architecture course — is bounded by an IPC ceiling of 1.0: at most one instruction completes per clock cycle, no matter how deeply the pipeline is staged. Modern CPUs, from Apple’s M-series to AMD’s Zen line, break this ceiling by executing multiple instructions per cycle and reordering their execution around data dependencies, while still presenting software with the illusion of simple, in-order execution. The mechanism that makes this possible — Tomasulo’s algorithm, introduced in 1967 for the IBM 360/91 — is the subject of this project.

PRAVAH is a 2-wide out-of-order superscalar RISC-V processor, implemented from scratch in synthesizable Verilog. The goal was not to build the fastest possible processor, but to correctly implement, integrate, and verify every mechanism that out-of-order execution depends on: register renaming to eliminate false dependencies, reservation stations for out-of-order dispatch, parallel execution units, and a reorder buffer to restore in-order commit semantics. This report documents the architecture, the design decisions behind it, the measured performance, and what was learned building it.

What was achieved: a working 2-wide OoO core that fetches, renames, dispatches, executes, and commits RV32I instructions, verified against a cycle-accurate testbench and synthesized on an Intel MAX 10 FPGA.

2. Overview

A scalar 5-stage pipeline (IF-ID-EX-MEM-WB) achieves at best $CPI = 1$ ($IPC = 1$) because each stage handles one instruction, and structural, data, control hazards routinely stall even that. Superscalar processors widen the pipeline to handle multiple instructions per stage, but naively doing so exposes new hazards: **false dependencies**. If instruction B writes the same architectural register that instruction A writes (WAW) or reads before A overwrites it

(WAR), the machine must preserve program-order semantics even though A and B may have no true data dependency.

Register renaming solves this by decoupling the architectural register namespace (what the ISA exposes) from a larger pool of physical registers (what the hardware actually has). Every instruction that writes a register is given a fresh physical register; a rename map tracks which physical register currently backs each architectural register. WAW and WAR hazards dissolve, because two instructions writing “the same” architectural register are actually writing different physical registers.

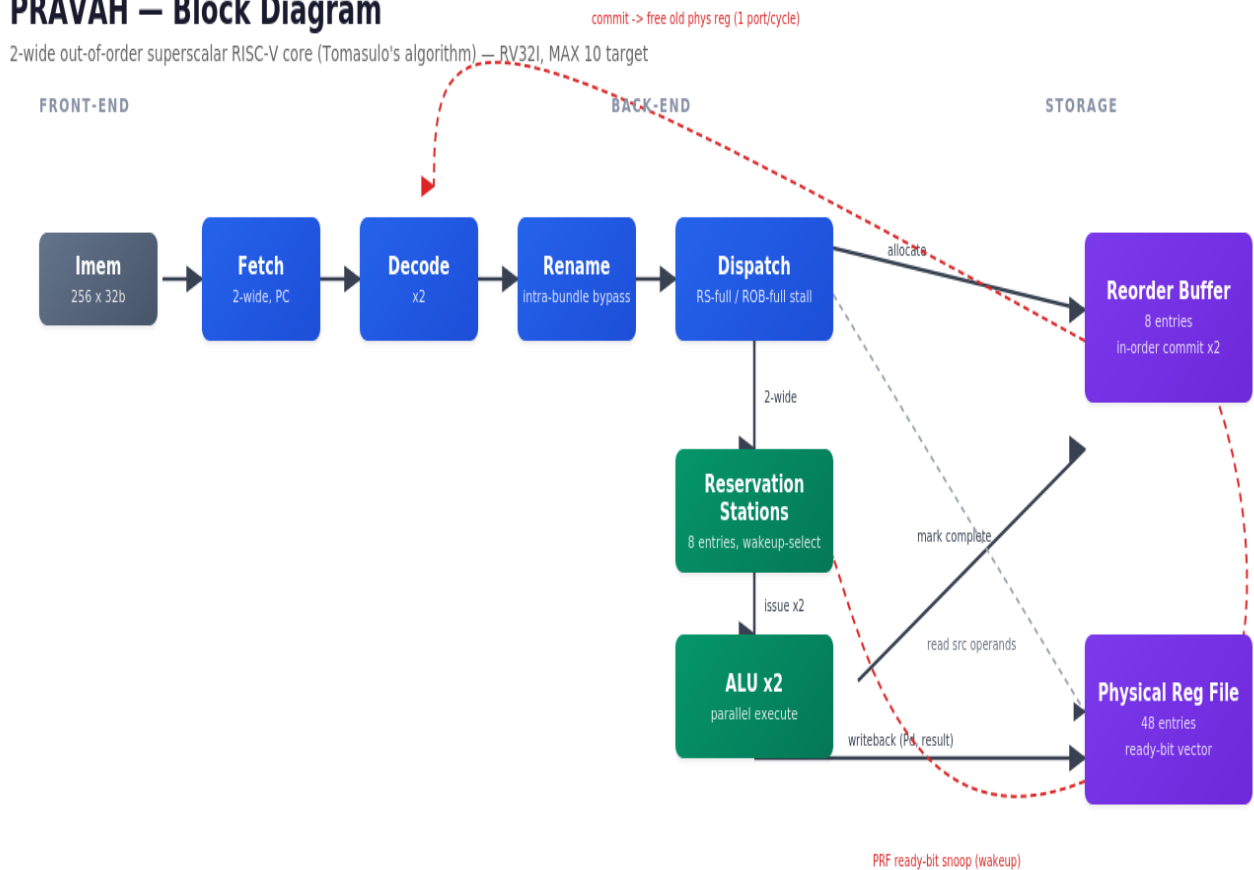
Renaming alone doesn’t enable out-of-order execution — it needs a place for instructions to wait for their (renamed) operands to become ready. **Reservation stations** hold dispatched-but-not-yet-executed instructions, tagging each source operand with either its ready value or a pointer to the physical register that will eventually produce it. When a functional unit finishes and writes back a result, every reservation station entry waiting on that physical register “wakes up,” and a **select** mechanism picks which of the now-ready entries actually issue this cycle.

Because instructions can now complete in any order, something has to restore program order for anything visible to software (register writeback, memory writes, exceptions). The **reorder buffer (ROB)** is a circular queue that tracks every in-flight instruction in program order; an instruction only **commits** — makes its result architecturally visible — when it reaches the head of the ROB and has finished executing, even if younger instructions finished earlier. This is Tomasulo’s algorithm.

3. Architecture

PRAVAH – Block Diagram

2-wide out-of-order superscalar RISC-V core (Tomasulo's algorithm) — RV32I, MAX 10 target



PRAVAH Block Diagram

3.1 Pipeline overview

PRAVAH's pipeline is: **Fetch (×2) → Decode (×2) → Rename → Dispatch → Reservation Stations → Execute (2× ALU) → Reorder Buffer (commit, ×2).**

Fetch reads two 32-bit instruction words per cycle from a 256-entry instruction memory, combinationally indexed by the current PC and PC+4.

Decode (×2) extracts the opcode, source/destination architectural register fields, and immediate for each of the two fetched instructions.

Rename maps each instruction's architectural source and destination registers to physical registers via a rename map, and allocates a fresh physical register for each destination from a 16-entry free list (48 physical registers total: 32 covering the architectural file at reset, 16 as the renaming pool). The rename stage handles the **intra-bundle dependency** case explicitly: if the second instruction in a fetch bundle reads a register that the first instruction in the *same* bundle writes, the second instruction must see the first instruction's newly-allocated physical register, not the old mapping — this requires a same-cycle bypass inside the rename logic rather than a simple sequential map lookup.

Dispatch allocates a ROB entry for each instruction (in program order) and inserts it into the reservation stations, stalling the entire 2-wide bundle if either the ROB or the reservation stations lack the required entries (`rob_full_i` / `rs_full_i`).

Reservation stations (8 entries, shared across both ALUs) hold dispatched instructions until their operands are ready. Readiness is tracked via a **wakeup** mechanism: each entry snoops a ready-bit vector from the physical register file, and a two-instruction-wide **select** stage picks the two lowest-index ready entries per cycle using a priority-encoder cascade (the second pick excludes whatever the first pick already selected).

Execution happens on two parallel ALUs, each computing a result combinationally within the cycle it issues.

The reorder buffer (8 entries) is the commit authority: it tracks every dispatched instruction in program order and releases results architecturally only when an instruction reaches the ROB head *and* has finished executing — enforcing in-order commit regardless of the order instructions actually executed in. On commit, the ROB signals the rename unit to return the instruction's old physical register to the free list.

3.2 Key structures

Structure	Parameters
Physical register file	48 entries, per-entry ready bit
Reservation stations	8 entries, 2-wide dispatch, 2-wide issue
Reorder buffer	8 entries, 2-wide commit (see §4.5 on the single commit port)
Instruction memory	256 × 32-bit words, combinational read
ALUs	2, single-cycle

4. Design Decisions

4.1 2-Wide, not 4-Wide

We chose a 2-wide front end (fetch/decode/rename/dispatch, 2 issue slots, 2 ALUs) rather than 4-wide. A wider machine exposes more parallelism per cycle, but the reservation station's wakeup-select cascade, the rename unit's intra-bundle checks, and the ROB's commit arbitration all scale worse than linearly with width. We accepted the lower theoretical ceiling (IPC = 2.0 rather than 4.0) because the project's aim was to correctly implement every OoO mechanism within an 8-week build, and 2-wide is sufficient to demonstrate all of them clearly.

4.2 48 Physical Registers

32 architectural registers plus a 16-entry renaming pool. A larger pool would reduce free-list exhaustion stalls at the cost of a wider physical register address threading through every RS entry, ROB entry, and rename-map entry. We accepted 48 as adequate headroom given the ROB itself caps in-flight instructions at 8 regardless of pool size.

4.3 8-Entry Reservation Stations, Shared Across Both ALUs

A single shared pool lets either ALU execute any ready instruction, avoiding the structural stalls of one ALU's dedicated RS filling while the other's sits empty — at the cost of arbitrating both issue slots across all 8 entries, producing two back-to-back priority-encoder cascades.

4.4 8-Entry Reorder Buffer

Matches the RS count, capping in-flight instructions at 8. A larger ROB would reduce dispatch stalls from a full ROB but widen ROB_ADDR_W throughout the design. We accepted 8 to keep the head/tail pointer arithmetic simple to verify.

4.5 Single commit port

We chose a single commit port for the rename unit's free-list return, rather than two. This significantly simplifies commit logic — one free-list write per cycle, no arbitration. The cost: when both ROB slots are committable in the same cycle, only one physical register is returned to the free list per cycle, throttling the free list's refill rate during commit-heavy phases and stalling dispatch. This directly shows up in the independent benchmark's IPC gap (§5.3). Widening to two commit ports is future work.

4.6 ALU-Only Datapath

We shipped an ALU-only baseline rather than attempting the optional MUL/LSU stretch goals. A multiplier introduces 3-writer/2-write-port CDB contention requiring arbitration and result-holding state; a load-store unit requires a store buffer and store-at-commit semantics. Both are substantial multi-day extensions. We prioritized a fully-tested, fully-understood ALU-only core over a partially-working extended one.

5. Results

5.1 IPC measurements

Benchmark	End-to-end IPC	Steady-state IPC
Independent	<u>1.6</u>	<u>2.0</u>
Dependency chain	<u>0.889</u>	<u>1.0</u>
Mixed	<u>1.4545</u>	<u>1.7778</u>

5.2 Synthesis results (Intel MAX 10, 10M50DAF484C7G)

Metric	Value
Fmax (Slow 1200mV 85C Model)	<u>85.8</u> MHz
Total logic elements	<u>332</u>
Total registers	<u>139</u>
Total memory bits	0
Critical path	RS wakeup-select loop

The critical path runs from the physical register file's ready-bit vector, through each reservation station entry's per-operand ready compare, through the two-deep priority-encoder select cascade (picking up to 2 ready entries out of 8), through the issue mux, through the ALU's combinational compute, and back to the physical register file's write port. This is the same fundamental bottleneck real out-of-order processors face — arbitrating which of several ready instructions gets to issue is inherently a wide combinational operation, and picking 2 per cycle (rather than 1) roughly doubles the logic depth of the select stage.

5.3 Gap analysis

The independent benchmark's IPC of 1.6 falls short of the 2.0 ceiling primarily due to the single commit port : when both ROB slots become committable together, only one physical register is freed per cycle, throttling the rate at which the free list can supply new destinations to dispatch. The chain's IPC of 0.889 sits close to the expected 1.0 floor — since every instruction depends on its immediate predecessor, the second issue slot is never used regardless of machine width, and the small shortfall below 1.0 is attributable to the 1-cycle wakeup-to-issue latency (a producer's result written back in cycle T can only wake its consumer in cycle T+1). The mixed benchmark's IPC of 1.7778 reflects the program's partial instruction-level parallelism — some sections behave like the independent case, others like the chain case.

6. Limitations:

- Single commit port throttles free-list return, measurably capping IPC on commit-heavy workloads (§5.3).
- ALU-only: no MUL (RV32M) or LW/SW (RV32I load/store) support.

- No branch prediction or speculative-state recovery; the design assumes straight-line code.
- Instruction memory synthesizes as LUT logic, not block RAM, due to its combinational read (inconsequential at 256 words, but would need revisiting at larger sizes).

7. Conclusion

PRAVAH implements the core mechanism behind every modern out-of-order processor — register renaming, out-of-order dispatch and execution via reservation stations, and in-order commit via a reorder buffer — in synthesizable Verilog, verified in simulation and synthesized for a real FPGA target. The measured results confirm the expected behavior: near-ceiling IPC on independent code, near-floor IPC on serial dependency chains, and a mid-range result on realistic mixed code, with every gap from the theoretical 2.0 ceiling attributable to a specific, identified architectural cause rather than unexplained overhead. Building and debugging the integration of eight interacting modules and tracing subtle bugs like floating ports silently propagating x through ready-bit logic was, in practice, harder than designing any single module in isolation, and is the most transferable lesson of the project.

8. References

- Hennessy, J. L., & Patterson, D. A. *Computer Architecture: A Quantitative Approach*.
- Lectures of Onur Mutlu and Smruti R. Sarangi — Basic and Advanced Lectures on Computer Architecture & Organisation (encapsulating Superscalar processors) , IIT Delhi.